ZeeshanAli-0704
Posted on Mar 12 • Edited on Mar 15

Frontend System Design : Frontend Security — Guide

#frontend #systemdesignwithzeeshanali #fsdzeeshan #interview

Frontend System Design (27 Part Series)

1 Frontend System Design: Facebook News Feed
2 Frontend System Design: CSS, CSSOM, and DOM Ren…
... **23 more parts...**
26 Frontend System Design: Performance Monitoring & Obs…
27 Frontend System Design: Virtualization & Handling Large…

Frontend Security — Frontend System Design Guide

> A comprehensive guide to security attack vectors, defense strategies, and secure architecture patterns from a **frontend engineer's perspective**.
>
> **See also:** Authentication Flows Guide — Session-based, JWT, OAuth 2.0, SSO, and PKCE.

Table of Contents

Security Attack Vectors and Prevention

1.1 XSS (Cross-Site Scripting)

What: Attacker injects malicious script into your page that runs in other users' browsers.

Types:

Type	How	Example
Stored XSS	Malicious script saved in DB, served to all users	Comment: `<script>steal(document.cookie)</script>`
Reflected XSS	Script in URL, reflected back in response	`site.com/search?q=<script>alert(1)</script>`
DOM-based XSS	Client-side JS writes untrusted data to DOM	`element.innerHTML = location.hash`

Prevention:

```
// ✖ DANGEROUS — never do this with user input
element.innerHTML = userInput;
document.write(userInput);

// ✔ SAFE — use textContent (auto-escapes HTML)
element.textContent = userInput;

// ✔ SAFE — React auto-escapes by default
function Comment({ text }) {
  return <p>{text}</p>;  // React escapes `text` automatically
}

// ✖ DANGEROUS — React escape hatch
<div dangerouslySetInnerHTML={{ __html: userInput }} />

// ✔ If you MUST render HTML, sanitize first
import DOMPurify from "dompurify";
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userInput) }} />
```

1.2 CSRF (Cross-Site Request Forgery)

What: Attacker tricks user's browser into making a request to your server, piggy-backing on existing cookies.

```
1. User is logged into bank.com (has session cookie)
2. User visits evil.com
3. evil.com has: <img src="https://bank.com/transfer?to=attacker&amount=1000">
4. Browser sends request WITH bank.com cookies → money transferred!
```

Prevention:

```
// 1. SameSite cookie attribute (best first line of defense)
res.cookie("session", sessionId, {
  sameSite: "lax",  // cookie NOT sent on cross-origin POST requests
  httpOnly: true,
  secure: true,
});

// 2. CSRF Token pattern
// Server generates a token, embeds it in the page
app.get("/form", (req, res) => {
  const csrfToken = generateCSRFToken();
  req.session.csrfToken = csrfToken;
```

```
// 3. Verify Origin / Referer header on the server
app.use((req, res, next) => {
  if (["POST", "PUT", "DELETE"].includes(req.method)) {
    const origin = req.headers.origin || req.headers.referer;
    if (!origin?.startsWith("https://yourapp.com")) {
      return res.status(403).json({ error: "Invalid origin" });
    }
  }
  next();
});
```

1.3 Clickjacking

What: Attacker embeds your site in an invisible iframe and tricks users into clicking.

```
<!-- Attacker's page -->
<style>
  iframe { opacity: 0; position: absolute; top: 0; left: 0; width: 100%; height: 100%; }
</style>
<h1>Click here to win a prize!</h1>
<iframe src="https://bank.com/transfer?to=attacker"></iframe>
<!-- User thinks they're clicking the prize button, but actually clicking inside the iframe -->
```

Prevention:

```
// Server: Set X-Frame-Options header
app.use((req, res, next) => {
  res.setHeader("X-Frame-Options", "DENY");          // never allow framing
  // OR
  res.setHeader("X-Frame-Options", "SAMEORIGIN");    // only same origin can frame
  next();
});

// Modern: Use CSP frame-ancestors (more flexible)
res.setHeader(
  "Content-Security-Policy",
  "frame-ancestors 'self'"  // only your own origin can embed this page
);

// Frontend: Frame-busting script (fallback)
if (window.top !== window.self) {
  window.top.location = window.self.location;
}
```

⬆ Back to Top

Content Security Policy (CSP)

CSP is an HTTP header that tells the browser **what resources are allowed to load**, preventing XSS and data injection.

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://cdn.example.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  connect-src 'self' https://api.example.com;
```

`img-src`	Image sources	`'self' data: https:`
`connect-src`	Fetch/XHR/WebSocket targets	`'self' https://api.example.com`
`font-src`	Font file sources	`'self' https://fonts.gstatic.com`
`frame-ancestors`	Who can embed your page (anti-clickjacking)	`'none'` or `'self'`
`base-uri`	Restricts `<base>` tag	`'self'`
`form-action`	Where forms can submit to	`'self'`
`upgrade-insecure-requests`	Auto-upgrade HTTP → HTTPS	*(no value needed)*

Code — Setting CSP with Helmet.js

```
const helmet = require("helmet");

app.use(
  helmet.contentSecurityPolicy({
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "https://cdn.example.com"],
      styleSrc: ["'self'", "'unsafe-inline'"],  // needed for many CSS-in-JS libs
      imgSrc: ["'self'", "data:", "https:"],
      connectSrc: ["'self'", "https://api.example.com"],
      fontSrc: ["'self'", "https://fonts.gstatic.com"],
      frameAncestors: ["'none'"],
      upgradeInsecureRequests: [],
    },
  })
);
```

CSP Nonce for Inline Scripts (Best Practice)

```
// Server generates a nonce per request
app.use((req, res, next) => {
  const nonce = crypto.randomBytes(16).toString("base64");
  res.locals.nonce = nonce;
  res.setHeader(
    "Content-Security-Policy",
    `script-src 'self' 'nonce-${nonce}'`
  );
  next();
});

// In your HTML template
app.get("/", (req, res) => {
  res.send(`
    <html>
      <body>
        <!-- Only scripts with the correct nonce will execute -->
        <script nonce="${res.locals.nonce}">
          console.log("This is allowed");
        </script>
      </body>
    </html>
  `);
});
```

CSP Reporting — Catch Violations in Production

```
Content-Security-Policy-Report-Only:
  default-src 'self';
```

CORS (Cross-Origin Resource Sharing) controls which origins can access resources from your server. The **browser** enforces this — not the server.

```
Same-Origin:    https://app.com     → https://app.com/api      ✅ Always allowed
Cross-Origin:   https://app.com     → https://api.app.com      ❌ Blocked unless CORS headers present
Cross-Origin:   https://app.com     → https://api.other.com    ❌ Blocked unless CORS headers present
```

What Makes a Request "Cross-Origin"?

Two URLs are different origins if **any** of these differ:

Component	Example A	Example B	Same Origin?
Protocol	`https://app.com`	`http://app.com`	❌
Domain	`https://app.com`	`https://api.app.com`	❌
Port	`https://app.com:443`	`https://app.com:8080`	❌
All same	`https://app.com/page1`	`https://app.com/page2`	✅

Simple vs Preflighted Requests

Simple Request	Preflighted Request
`GET`, `HEAD`, `POST`	`PUT`, `DELETE`, `PATCH`
Only standard headers (`Accept`, `Content-Type` with form types)	Custom headers (`Authorization`, `X-CSRF-Token`)
No preflight needed	Browser sends `OPTIONS` first

Preflight Request (for "complex" requests)

```
Browser                          Server
  |                                |
  |──OPTIONS /api/data ──────────▶|   ← Preflight check
  |  Origin: https://app.com      |
  |  Access-Control-Request-Method: POST
  |  Access-Control-Request-Headers: Authorization, Content-Type
  |                                |
  |◀─ 204 No Content ─────────────|   ← Server responds with allowed origins/methods
  |  Access-Control-Allow-Origin: https://app.com
  |  Access-Control-Allow-Methods: GET, POST
  |  Access-Control-Allow-Headers: Authorization, Content-Type
  |  Access-Control-Max-Age: 86400
  |                                |
  |──POST /api/data ─────────────▶|   ← Actual request (if preflight passed)
  |  Origin: https://app.com      |
```

Code — Express CORS Setup

```
const cors = require("cors");

// ❌ BAD — allows everything (fine for dev, terrible for production)
app.use(cors());

// ✅ GOOD — specific origins
app.use(
  cors({
    origin: ["https://app.com", "https://staging.app.com"],
    methods: ["GET", "POST", "PUT", "DELETE"],
    allowedHeaders: ["Content-Type", "Authorization"],
```

```
    credentials: true,
  })
);
```

Common CORS Mistakes

Mistake	Why It's Bad
`Access-Control-Allow-Origin: *` with `credentials: true`	Browser **rejects** this combination
Forgetting `credentials: "include"` on fetch	Cookies won't be sent cross-origin
Not handling OPTIONS preflight	Browser blocks actual request
Reflecting any `Origin` header as allowed	Same as `*` — defeats the purpose

⬆ Back to Top

Secure Cookie Handling

```
res.cookie("session", token, {
  httpOnly: true,       // ← JS cannot access (prevents XSS token theft)
  secure: true,         // ← only sent over HTTPS
  sameSite: "strict",   // ← not sent on ANY cross-origin request
  maxAge: 30 * 60 * 1000, // 30 minutes
  domain: ".app.com",   // ← shared across subdomains
  path: "/",
});
```

Cookie Flags Explained

Flag	Purpose	Value
`HttpOnly`	Prevents `document.cookie` access	`true` — always for auth cookies
`Secure`	Only sent over HTTPS	`true` — always in production
`SameSite`	Controls cross-origin cookie sending	`Strict` / `Lax` / `None`
`Domain`	Which domains receive the cookie	`.app.com` for subdomain sharing
`Path`	Cookie scope within the domain	`/` or `/api`
`Max-Age`	TTL in seconds	1800 (30 min)

SameSite Values

Value	Behavior	CSRF Protection
`Strict`	Never sent cross-origin (safest, but breaks some UX like link clicks)	✅ Strong
`Lax`	Sent on **top-level GET** navigations only (good default)	✅ Good
`None`	Always sent cross-origin (**requires** `Secure: true`)	❌ None

Cookie Security Decision Tree

```
Is it an auth cookie (session ID, token)?
  |
```

Rule of Thumb

> **Sanitize input. Encode output. Trust nothing from the user.**

```
// ——— INPUT SANITIZATION ———

// 1. DOMPurify for HTML
import DOMPurify from "dompurify";
const cleanHTML = DOMPurify.sanitize(userInput);
// "<img onerror=alert(1) src=x>" → "<img src=\"x\">"

// 2. Validate & constrain input
function validateEmail(email) {
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!re.test(email)) throw new Error("Invalid email");
  if (email.length > 254) throw new Error("Email too long");
  return email.trim().toLowerCase();
}

// 3. Parameterized queries (backend — prevent SQL injection)
// ✖ db.query(`SELECT * FROM users WHERE id = ${userId}`);
// ✔ db.query("SELECT * FROM users WHERE id = $1", [userId]);


// ——— OUTPUT ENCODING ———

// 1. HTML context — escape entities
function escapeHTML(str) {
  const map = { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" };
  return str.replace(/[&<>"']/g, (ch) => map[ch]);
}

// 2. URL context — encode components
const searchUrl = `/search?q=${encodeURIComponent(userQuery)}`;

// 3. JSON context — JSON.stringify handles encoding
const jsonSafe = JSON.stringify({ name: userInput });
```

Where to Sanitize — Frontend vs Backend

	Frontend	Backend
Purpose	UX — prevent accidental rendering of HTML	Security — never trust client input
Example	DOMPurify before rendering	Validate + sanitize before storing in DB
Can be bypassed?	✔ Yes (user can modify JS, use curl)	✖ No (server-controlled)
Verdict	Defense in depth — helpful, not sufficient	**Mandatory** — the real security boundary

> **Golden Rule:** Always sanitize on the **backend**. Frontend sanitization is a bonus, not a replacement.

⬆ Back to Top

JWT Security Considerations

JWTs are widely used for authentication, but they come with unique security concerns. *(For JWT concepts and auth flows, see Authentication Flows Guide.)*

Where to Store JWT on the Frontend?

Storage	XSS Safe?	CSRF Safe?	Recommendation

Storing JWT in `localStorage`	XSS can steal the token	Use in-memory or HttpOnly cookie
No expiry (`exp` claim)	Token valid forever if leaked	Always set short `expiresIn` (15 min)
Using `alg: none`	Signature verification bypassed	Reject `none` algorithm on server
Same secret for access + refresh tokens	Compromising one compromises both	Use different secrets/keys
Not validating `iss` and `aud` claims	Token from another app is accepted	Always verify issuer and audience
Putting sensitive data in payload	Anyone can decode JWT (it's base64, not encrypted)	Only store non-sensitive claims
Not implementing token revocation	Can't invalidate a compromised token	Maintain a blocklist or use short expiry + refresh tokens

JWT Refresh Token Security

```
Access Token (short-lived: 15 min)
  ├── Stored in memory (JS variable)
  ├── Sent via Authorization header
  └── If stolen → limited damage (expires soon)

Refresh Token (long-lived: 7 days)
  ├── Stored in HttpOnly secure cookie
  ├── Sent only to /refresh endpoint (path-scoped cookie)
  ├── Server-side: implement rotation (new refresh token each use)
  └── If stolen → attacker gets new access tokens until detected
```

Refresh Token Rotation — each time a refresh token is used, the server issues a **new** refresh token and invalidates the old one. If an attacker replays an already-used refresh token, the server knows it was stolen and revokes the entire token family.

⬆ Back to Top

Frontend Auth Architecture Patterns

Pattern 1: Auth Provider + Protected Routes (React)

```jsx
// App.jsx — complete auth flow wiring
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider, useAuth } from "./AuthContext";

function ProtectedRoute({ children, requiredRole }) {
  const { user, isLoading } = useAuth();

  if (isLoading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (requiredRole && user.role !== requiredRole) {
    return <Navigate to="/unauthorized" replace />;
  }
  return children;
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<LoginPage />} />
          <Route path="/dashboard" element={
```

Pattern 2: Axios Interceptor for Token Management

```
// api.js — centralized auth handling
import axios from "axios";

const api = axios.create({
  baseURL: "https://api.yourapp.com",
  withCredentials: true,
});

let accessToken = null;
let isRefreshing = false;
let failedQueue = [];

const processQueue = (error, token = null) => {
  failedQueue.forEach(({ resolve, reject }) => {
    error ? reject(error) : resolve(token);
  });
  failedQueue = [];
};

// Attach token to every request
api.interceptors.request.use((config) => {
  if (accessToken) {
    config.headers.Authorization = `Bearer ${accessToken}`;
  }
  return config;
});

// Auto-refresh on 401
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    if (error.response?.status !== 401 || originalRequest._retry) {
      return Promise.reject(error);
    }

    if (isRefreshing) {
      // Queue failed requests while refreshing
      return new Promise((resolve, reject) => {
        failedQueue.push({ resolve, reject });
      }).then((token) => {
        originalRequest.headers.Authorization = `Bearer ${token}`;
        return api(originalRequest);
      });
    }

    originalRequest._retry = true;
    isRefreshing = true;

    try {
      const { data } = await axios.post("/auth/refresh", null, {
        withCredentials: true,
      });
      accessToken = data.accessToken;
      processQueue(null, accessToken);
      originalRequest.headers.Authorization = `Bearer ${accessToken}`;
      return api(originalRequest);
    } catch (err) {
      processQueue(err);
      accessToken = null;
      window.location.href = "/login";
      return Promise.reject(err);
    } finally {
      isRefreshing = false;
    }
```

```
    viewer: ["read"],
  };

  const userPerms = permissions[user?.role] || [];
  const allowed = userPerms.includes(perform);

  return allowed ? children : fallback;
}

// Usage
function PostActions({ post }) {
  return (
    <div>
      <Can perform="update" on="post">
        <button onClick={() => editPost(post.id)}>Edit</button>
      </Can>
      <Can perform="delete" on="post">
        <button onClick={() => deletePost(post.id)}>Delete</button>
      </Can>
      <Can perform="delete" on="post" fallback={<span>View only</span>}>
        <button>Manage</button>
      </Can>
    </div>
  );
}
```

⬆ Back to Top

Iframe Security

Iframes are powerful but introduce significant security risks. As a frontend engineer, you need to understand both **protecting your app from being iframed** and **protecting your app when embedding third-party iframes**.

Two Sides of Iframe Security

```
┌─────────────────────────────────────────────────────┐
│                  Iframe Security                    │
│                                                     │
│  1. DEFENDING your app          2. EMBEDDING third-party │
│     from being iframed             content safely   │
│                                                     │
│  Your site in attacker's        Third-party in your │
│  iframe → Clickjacking          iframe → XSS, data leak │
│                                                     │
│  Fix: X-Frame-Options,          Fix: sandbox attribute, │
│  CSP frame-ancestors            CSP frame-src       │
└─────────────────────────────────────────────────────┘
```

Side 1: Preventing Your App from Being Iframed (Clickjacking Defense)

If an attacker embeds your site in their iframe, they can overlay invisible UI to trick users into clicking buttons on your site (clickjacking).

Defense	How	Browser Support
`X-Frame-Options: DENY`	Blocks ALL framing of your page	✅ All browsers
`X-Frame-Options: SAMEORIGIN`	Only same-origin can frame	✅ All browsers
`CSP: frame-ancestors 'self'`	Modern replacement — more flexible	✅ Modern browsers
`CSP: frame-ancestors 'none'`	Blocks ALL framing (like DENY)	✅ Modern browsers

Side 2: Safely Embedding Third-Party Content in Your App

When **you** embed an iframe (e.g., a payment widget, video player, third-party form), the iframe can potentially:

- Run arbitrary JavaScript
- Access your parent page (if same-origin)
- Trigger navigation (redirect your page)
- Open popups
- Submit forms
- Access camera/microphone

The `sandbox` Attribute — Your Primary Defense

The `sandbox` attribute on `<iframe>` **restricts everything by default** and lets you selectively re-enable capabilities:

```
<!-- Maximum restriction — iframe can do almost nothing -->
<iframe src="https://third-party.com/widget" sandbox></iframe>

<!-- Selectively allow specific capabilities -->
<iframe
  src="https://third-party.com/widget"
  sandbox="allow-scripts allow-forms allow-same-origin"
></iframe>
```

`sandbox` Value	What It Allows	Risk Level
(empty — no value)	Nothing — fully sandboxed	✓ Safest
`allow-scripts`	JavaScript execution	⚠ Medium
`allow-forms`	Form submission	⚠ Medium
`allow-same-origin`	Treats iframe as same-origin (access cookies, storage)	High
`allow-popups`	Opening new windows/tabs	⚠ Medium
`allow-top-navigation`	Redirecting the parent page	High
`allow-modals`	`alert()`, `confirm()`, `prompt()`	Low

> ⚠ **Never combine `allow-scripts` + `allow-same-origin`** on untrusted content — the iframe script could remove the sandbox attribute from itself and break out of the sandbox entirely.

CSP `frame-src` — Control Which URLs Can Be Iframed

```
Content-Security-Policy: frame-src 'self' https://www.youtube.com https://player.vimeo.com;
```

This tells the browser: "Only allow iframes pointing to these sources. Block everything else."

CSP Directive	Controls
`frame-src`	What URLs can appear in `<iframe>` and `<frame>` on your page
`frame-ancestors`	What URLs can embed **your** page in their iframe (the opposite direction)
`child-src`	Fallback for `frame-src` + web workers

Cross-Origin Communication with Iframes (`postMessage`)

When you legitimately need to communicate between your page and an iframe (e.g., a payment widget telling you payment succeeded):

```
    handlePaymentSuccess(event.data.transactionId);
  }
});
```

`postMessage` Security Rules:

Rule	Why
Always specify target origin (never use `"*"`)	Prevents leaking data to wrong iframe if src changes
Always validate `event.origin` in the listener	Prevents any malicious iframe from sending fake messages
Validate `event.data` **structure**	Don't trust arbitrary shapes — treat it like user input
Never `eval()` **or** `innerHTML` **data from postMessage**	It's untrusted input — XSS risk

Iframe Security Decision Tree

```
Do you NEED an iframe?
  │
  ├── NO → Don't use one. Embed via API or server-side rendering instead.
  │
  └── YES → Is it YOUR content (same origin)?
        │
        ├── YES → Use `sandbox="allow-scripts allow-same-origin"`
        │         + CSP frame-src 'self'
        │
        └── NO → Is it a TRUSTED third party (Stripe, YouTube)?
              │
              ├── YES → sandbox="allow-scripts allow-forms"
              │         + CSP frame-src with their specific domain
              │         + postMessage with origin validation
              │
              └── NO (untrusted) → sandbox (empty, maximum restriction)
                    + CSP frame-src with their specific domain
                    + NO allow-same-origin
                    + NO allow-top-navigation
```

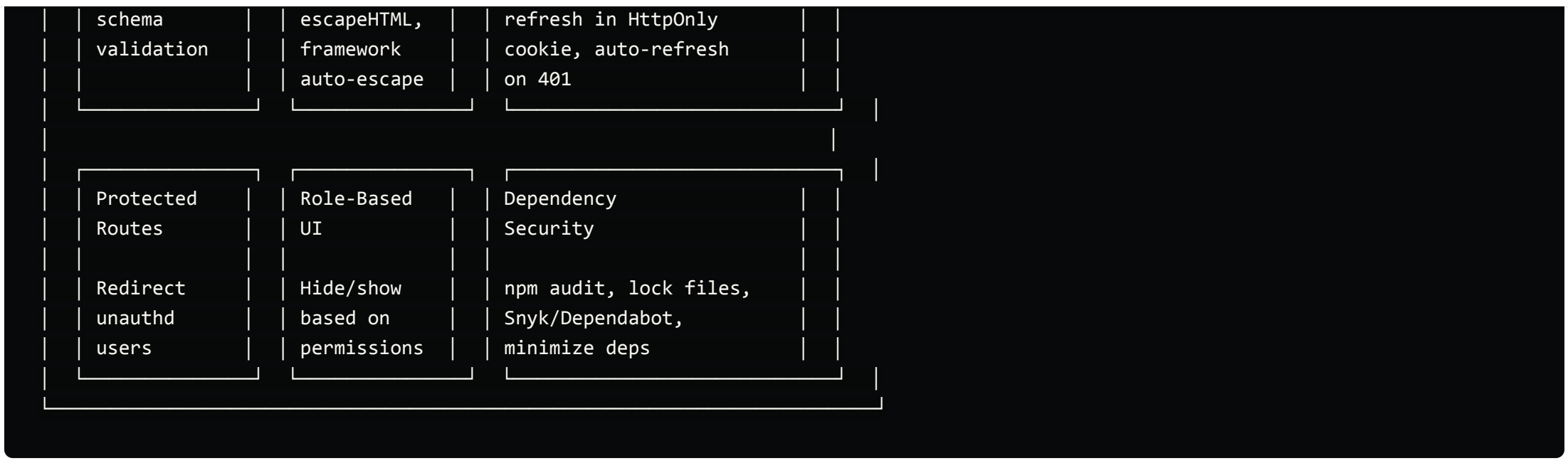
⬆ Back to Top

Overall Frontend Security Big Picture

Frontend security is **defense in depth** — no single measure is enough. Here's how all the pieces fit together:

The Frontend Security Layers

```
┌──────────────────────────────────────────────────────────────┐
│                      BROWSER LAYER                           │
│                                                              │
│  ┌─────────────────┐  ┌─────────────────┐  ┌──────────────────────────────┐  │
│  │ Same-Origin     │  │ CSP             │  │ Cookie Security              │  │
│  │ Policy (SOP)    │  │ Headers         │  │ (HttpOnly, Secure,           │  │
│  │                 │  │                 │  │  SameSite)                   │  │
│  │ Built-in:       │  │ You config:     │  │                              │  │
│  │ blocks cross-   │  │ whitelist       │  │ Prevents:                    │  │
│  │ origin reads    │  │ allowed         │  │ • XSS token theft            │  │
│  │                 │  │ resources       │  │ • CSRF                       │  │
│  └─────────────────┘  └─────────────────┘  └──────────────────────────────┘  │
│                                                              │
│  ┌─────────────────┐  ┌─────────────────┐  ┌──────────────────────────────┐  │
│  │ CORS            │  │ Iframe          │  │ HTTPS (HSTS)                 │  │
```



Attack → Defense Map

Attack	What Happens	Primary Defense	Secondary Defense
XSS (Stored/Reflected)	Attacker runs JS in user's browser	CSP headers	Input sanitization, output encoding, framework auto-escape
XSS (DOM-based)	Client-side JS injects untrusted data	Avoid `innerHTML`, use `textContent`	DOMPurify, CSP
CSRF	Forged request with victim's cookies	`SameSite` cookie attribute	CSRF tokens, Origin header validation
Clickjacking	Your site framed invisibly	`X-Frame-Options` / `CSP frame-ancestors`	Frame-busting JS
Man-in-the-Middle	Attacker intercepts HTTP traffic	HTTPS + HSTS header	`Secure` cookie flag
Token Theft (via XSS)	JS reads tokens from storage	`HttpOnly` cookies (JS can't access)	Store access token in memory only
Iframe Injection	Malicious iframe embedded in your page	CSP `frame-src`	`sandbox` attribute
Open Redirect	User redirected to phishing site	Validate redirect URLs server-side	Whitelist allowed domains
Dependency Attack	Malicious code in npm package	`npm audit`, lock files	Snyk/Dependabot, review new deps
Prototype Pollution	Attacker modifies `Object.prototype`	`Object.freeze(Object.prototype)`	Input validation, avoid `_.merge` with user data

Dependency Security — The Forgotten Vector

Your app is only as secure as its **weakest dependency**. A single compromised npm package can steal user data from every app that imports it.

Practice	How	Why
Run `npm audit` regularly	`npm audit --production`	Finds known vulnerabilities in dependencies
Use lock files	Commit `package-lock.json`	Prevents unexpected version changes
Enable Dependabot / Snyk	GitHub Settings → Security	Auto-creates PRs for vulnerable deps
Minimize dependencies	Ask: "Do I really need this package?"	Fewer deps = smaller attack surface

```
  src="https://cdn.example.com/lib.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxIqF32UWh4j7Jo0Fx7gWFQhJy3oE"
  crossorigin="anonymous"
></script>
```

How it works: The `integrity` attribute contains a cryptographic hash of the expected file. If the file served by the CDN doesn't match, the browser **refuses to execute it**.

Frontend Security Audit Checklist

```
☐ Security Headers
  ☐ CSP configured and enforced (not just report-only)
  ☐ HSTS enabled with includeSubDomains
  ☐ X-Frame-Options set
  ☐ X-Content-Type-Options: nosniff
  ☐ Referrer-Policy configured
  ☐ Permissions-Policy restricts unused APIs

☐ XSS Prevention
  ☐ No innerHTML with user input
  ☐ No dangerouslySetInnerHTML without DOMPurify
  ☐ CSP blocks inline scripts (or uses nonces)
  ☐ All user input validated + sanitized on backend

☐ Auth & Tokens
  ☐ Access token in memory (not localStorage)
  ☐ Refresh token in HttpOnly secure cookie
  ☐ Token refresh logic handles race conditions
  ☐ Logout clears all tokens and sessions

☐ Cookies
  ☐ Auth cookies: HttpOnly + Secure + SameSite
  ☐ Session cookies have reasonable maxAge
  ☐ No sensitive data in non-HttpOnly cookies

☐ CORS
  ☐ Specific origin whitelist (not *)
  ☐ credentials: true only with explicit origins
  ☐ OPTIONS preflight handled correctly

☐ Iframes
  ☐ Third-party iframes use sandbox attribute
  ☐ CSP frame-src restricts allowed iframe sources
  ☐ postMessage validates event.origin
  ☐ No allow-scripts + allow-same-origin on untrusted content

☐ Dependencies
  ☐ npm audit clean (no critical/high vulnerabilities)
  ☐ Lock file committed and up-to-date
  ☐ CDN scripts use SRI (integrity attribute)
  ☐ Dependabot / Snyk enabled

☐ HTTPS
  ☐ All traffic over HTTPS
  ☐ HSTS header set
  ☐ No mixed content (HTTP resources on HTTPS page)
```

⬆ Back to Top

Security Headers Checklist

Every production frontend app should ensure these headers are set by the server:

X-Content-Type-Options	Prevents MIME type sniffing	Browser executes mistyped resources
X-Frame-Options	Prevents your page from being framed	Clickjacking
Referrer-Policy	Controls what URL info is sent to other sites	Leaking sensitive URL paths
Permissions-Policy	Disables browser features you don't use	Malicious scripts accessing camera/mic

⬆ Back to Top

Interview Cheat Sheet

Quick Answers for System Design Interviews

Q: How do you prevent XSS?

> CSP headers, avoid `innerHTML`/`dangerouslySetInnerHTML`, sanitize with DOMPurify, use `textContent`, rely on framework auto-escaping.

Q: How do you prevent CSRF?

> `SameSite=Lax` or `Strict` cookies, CSRF tokens for state-changing requests, verify `Origin`/`Referer` headers.

Q: What is CSP and why is it important?

> Content Security Policy is an HTTP header that whitelists allowed resource sources. It's the strongest defense against XSS — even if an attacker injects a script tag, the browser won't execute it if the source isn't allowed.

Q: Explain CORS in simple terms.

> The browser blocks frontend JS from reading responses from a different origin unless the server explicitly allows it via `Access-Control-Allow-Origin` headers. It's a browser-enforced policy, not a server-side firewall.

Q: Where should I store tokens on the frontend?

> Access token in **memory** (JS variable). Refresh token in an **HttpOnly, Secure, SameSite cookie**. Never use `localStorage` for auth tokens.

Q: What's the difference between `HttpOnly`, `Secure`, and `SameSite`?

> `HttpOnly` = JS can't read the cookie (XSS protection). `Secure` = only sent over HTTPS. `SameSite` = controls when cookies are sent cross-origin (CSRF protection).

Q: How would you audit the security of a frontend app?

> 1. Check security headers (CSP, HSTS, X-Frame-Options). 2. Search for `innerHTML`/`dangerouslySetInnerHTML` usage. 3. Verify token storage (no `localStorage` for auth). 4. Check CORS config. 5. Run automated tools (Lighthouse, OWASP ZAP). 6. Review cookie flags.

Q: How do you secure iframes?

> Two directions: (1) Prevent **your** site from being iframed via `X-Frame-Options: DENY` and `CSP: frame-ancestors`. (2) When **embedding** third-party iframes, use the `sandbox` attribute to restrict capabilities and `CSP: frame-src` to whitelist allowed sources. Always validate `event.origin` when using `postMessage`.

Q: What is Subresource Integrity (SRI)?

> SRI is an `integrity` attribute on `<script>` and `<link>` tags that contains a hash of the expected file. If a CDN is compromised and serves tampered code, the browser checks the hash and **refuses to execute** it.

Q: How do you handle dependency security?

systemdesignwithzeeshanali

Git: https://github.com/ZeeshanAli-0704/front-end-system-design

⬆ Back to Top

Frontend System Design (27 Part Series)

1 Frontend System Design: Facebook News Feed

2 Frontend System Design: CSS, CSSOM, and DOM Ren…

... **23 more parts...**

26 Frontend System Design: Performance Monitoring & Obs…

27 Frontend System Design: Virtualization & Handling Large…

Top comments (0)

Subscribe

Kindness is contagious

Dive into this thoughtful piece, beloved in the supportive DEV Community. **Coders of every background** are invited to share and elevate our collective know-how.

A sincere "thank you" can brighten someone's day—leave your appreciation below!

On DEV, **sharing knowledge smooths our journey** and tightens our community bonds. Enjoyed this? A quick thank you to the author is hugely appreciated.

Okay

ZeeshanAli-0704

Follow

Results-driven Principal Applications Engineer with 9+ years of experience in scalable web app development using React, Angular, Node.js, and KnockoutJS across BFSI, Media, and Healthcare domains.

LOCATION
INDIA

PRONOUNS
he

WORK
Microsoft

JOINED
Aug 13, 2022

More from ZeeshanAli-0704

Frontend System Design: Virtualization & Handling Large Data Sets
#systemdesignwithzeeshanali #frontend #interview #fsdzeeshan

Frontend System Design: Performance Monitoring & Observability

AWS Builder Center

Connect with builders who understand your journey. Share solutions, influence AWS product development, and access useful content that accelerates your growth.